

packages, you must build them manually from source. Sounds daunting, but the earlier-mentioned slackbuilds.org website contains a large collection of *slackbuilds* (build scripts for Slackware packages) to aid you. You can manually download the scripts and run them, or use the *shpkg* tool to automatically compile and install packages from slackbuilds.org. The huge size of Slackware works in favour of users, since a large number of libraries are pre-installed. This explains how Slackware gets away without huge dependency headaches when you install new applications; chances are that they are already installed on your system!

Lastly, Slackware is a well-tested and stable Linux distribution, which is flexible and secure enough, and equally ready for use in the home or in the office.

Cons

Due to its 'vanilla' nature, Slackware might fail to attract a certain class of users who rate the default looks and appearance higher than usefulness and stability.

Even though it is possible to mould and shape a Linux OS into any form you like, that theory seldom holds true in practice, especially for newbies and those without patience or time! Even though it is rare to have a dependency problem, it's not unheard of. Since slackbuilds.org is an unofficial user repository, there may sometimes be cases where a package has not been updated to the latest version—but the packages it depends on have been updated by some other maintainer. This doesn't mean all is chaos; you just manually change it to the updated source and MD5 sums, and you may also have to modify a few things in the *slackbuild* before you can make it play nicely with other packages. It has been observed that often, *slackbuilds* do not compile, since the sources have changed and the older files don't exist any more (slackbuilds.org only stores the *slackbuilds* and other package description files, and not the actual source packages). Here, the user again has to update the *slackbuild* manually—not impossible, but certainly tedious and time consuming.

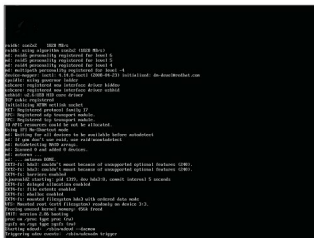
Another problem is lack of contribution. Many packages are not present in Slackware repositories or even as *slackbuilds*. It becomes very difficult to hunt down such packages, along with their dependencies, one by one.

Slackware being a well-tested system, uses only packages that have been proven stable. If you are willing to use slightly out-dated packages, but do not want to compromise on stability, then this is not a concern. Others who want the latest versions of packages may update to the testing or the 'current' branch, which usually undergoes heavy testing until packages are stable enough to be pushed into the next stable release. The frequency of Slackware releases is unpredictable; it is released 'when it's ready'.

Help and support makes a vast difference for any software platform; though Slackware has various forums,



Slackware LILO boot prompt



Default Slackware boot process – No Plymouth, no fun!

IRC, mailing lists, etc, there's still a problem here: often, a newbie (lacking knowledge of Linux and its functioning, perhaps on account of moving from another operating system) with a simple question is looked down on in the forums or on IRC. This attitude of the "experts" can be demeaning and hurtful at times, and can chase away potential users. Obviously this varies based on the individuals providing support, but such an elitist demeanour does exist; people expect users to know about their problems, and show proof of their efforts to fix the problems. Silly questions are trashed and mocked. In contrast, for user-friendly distributions like Ubuntu, Mandriva, OpenSUSE, etc, the attitude in the help and support channels is friendly, and even new users are treated with respect and kindness.

Tackling the dependency issue

The easiest way to check for dependencies is to run `ldd` (which shows shared library dependencies) on whatever you want to install. If you are installing something from slackbuilds.org, they list any required dependencies on the